

Dynamic Quality-of-Service for Mixed Workloads in a Data-Sovereign Private Cloud

Bachelor Thesis

Michael Voemel

29. June 2026

ZHAW School of Engineering · Institute of Computer Science
Supervisor: Dr. Leonardo Militano

Agenda

Motivation & Problem

Approach

Results

Discussion & Conclusion

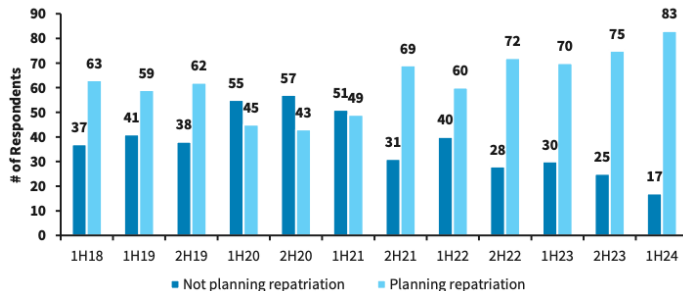
Motivation & Problem

Approach

Results

Discussion & Conclusion

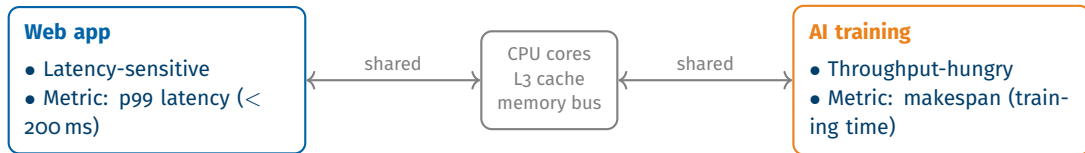
Why private cloud, why now



- Two drivers of **repatriation**: unpredictable public-cloud cost, and data sovereignty.
- Swiss **nFADP** plus the US **CLOUD Act**: a Swiss-located hyperscaler datacenter is not enough (the provider's corporate domicile still exposes the data).
- ⇒ for the most sensitive data, only **fully self-hosting** removes the provider from the picture.

Repatriation (83 % of CIOs): Barclays survey (2024); data sovereignty: nFADP / CLOUD Act. Report §1.1, §2.1.

The mixed-workload reality



No limits → web latency collapses, **SLO violation**

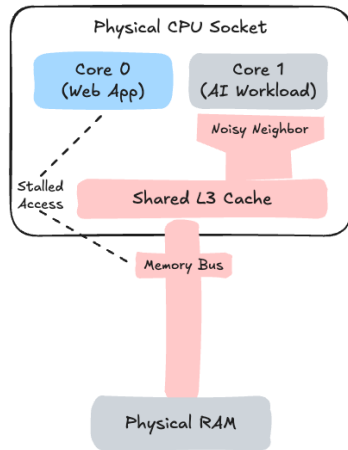
Static limits → idle CPU stranded, **wasted capacity**

Neither extreme wins → this motivates a **dynamic** middle ground.

Workload co-location for efficiency: Zhang et al. (2021); noisy neighbor: Thota (2025). Report §2.4.

The noisy-neighbor problem

- Even with cgroup core-pinning, the workloads share **physical** hardware: L3 cache, memory bus, NIC.
- AI tensors saturate the **memory bus** → the web app's RAM fetch stalls → latency spikes.
- Partitioning cores does not partition the shared bus. The fix: **dynamically throttle** the AI's CPU bandwidth exactly when the web app needs headroom.



Noisy neighbor: Thota (2025), Zhang et al. (2021). Report §2.4.

Can a feedback-driven PI controller dynamically reclaim idle CPU for a co-located AI workload while protecting the HTTP latency SLO of a web app on the same bare-metal Kubernetes nodes?

Protect the SLO ↻ Maximize AI throughput ⇒ one in-cluster loop

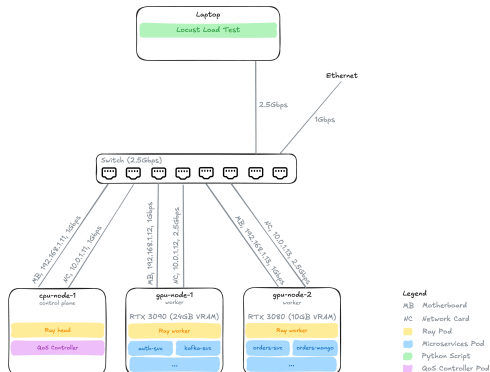
Motivation & Problem

Approach

Results

Discussion & Conclusion

System overview



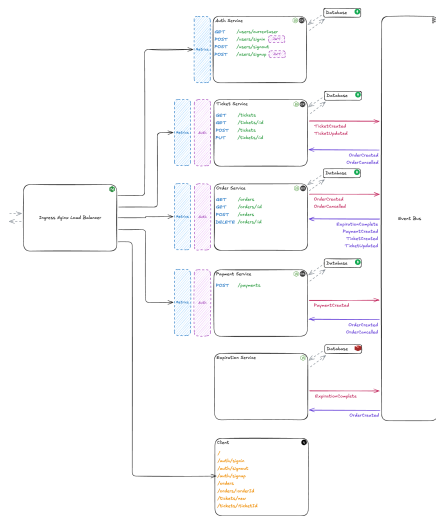
- 3-node **bare-metal** Kubernetes v1.34:
cpu-node-1 = control plane + Ray head;
gpu-node-1 (RTX 3090) & gpu-node-2 (RTX 3080) = AI workers + web replicas.
- **Ansible** IaC: fully reproducible, idempotent provisioning.
- Both workloads **deliberately co-located** on the two GPU nodes.

Bare-metal Kubernetes, Ansible IaC; both workloads co-located. Report §3.1.

Workload A: the Ticket App

Latency-sensitive web app, the SLO subject.

- 6 microservices, **Kafka-only**, MongoDB per data service (Redis for expiration).
- tickets & orders run **2 replicas** each → the busiest services.
- **PBKDF2 token** per order (42k SHA-256) lifts node CPU ~5 % → ~16–20 %; makes contention measurable.
- payments **excluded** from the SLO (500 ms mock delay masks the signal).
- SLA = $P_{95} \leq 150$ ms (controller). SLO = $P_{99} < 200$ ms (eval).

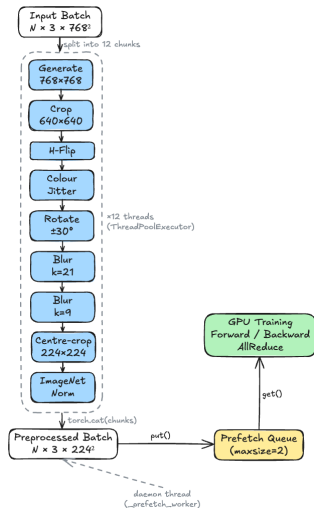


Microservices: Newman (2021). Report §3.3.

Workload B: the Ray AI job

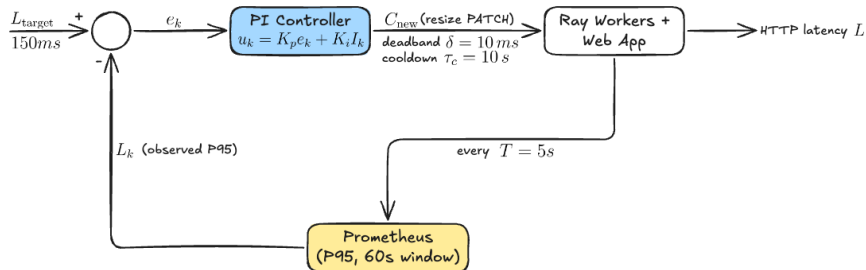
The noisy neighbor, throughput-heavy AI job.

- ResNet-50 (50 layer deep CNN), PyTorch **DDP**, 2 GPU workers via **Ray**.
- Deliberately **CPU-bottlenecked**: 9-stage preprocessing on 12 threads + prefetch queue, not GPU-bound.
- Each step also syncs gradients over the network (**NCCL AllReduce**), but that traffic is small → **the network is not the bottleneck**.
- The Ray Workers CPU cap is the **controller's lever**; **makespan** (total training time) is how its effect is measured.



Ray: Moritz et al. (2018); DDP: Li et al. (2020); preprocessing: Isenko et al. (2022). Report §3.4.

The QoS controller: the control loop



Observe (P95 from Prometheus) → **Analyze** (PI law) → **Act** (PATCH Ray CPU limit), every 5 s.

```
histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket{
  namespace="ticket-app", service!="payments-srv"}[60s])) by (le)) * 1000
```

PI control: Abranches & Solis (2016); KOSMOS: Baresi et al. (2021). Report §3.5.1.

The PI control law

$$e_k = L_k - L_{\text{target}}$$

$$I_k = I_{k-1} + e_k \cdot T$$

$$u_k = K_p e_k + K_i I_k$$

$$C_{\text{new}} = \text{clamp}(\text{round}(C_{\text{current}} - u_k), C_{\text{min}}, C_{\text{max}})$$

K_p : immediate, linear reaction to the current error. K_i : accumulates past error $\rightarrow$ erases the steady-state offset.

Sign: $e_k > 0 \rightarrow$ **reduce** Ray CPU (throttle); $e_k < 0 \rightarrow$ restore. Skip if $|e_k| < 10$ ms (deadband); integral frozen when saturated (anti-windup).

Worked example

$L_k=250$ ms $\rightarrow e_k=100$ ms; with $I_k=500$ ms·s: $u_k = 0.02 \cdot 100 + 0.0002 \cdot 500 = 2.0 + 0.1 = 2.1$ cores. From $C_{\text{current}}=8$: $C_{\text{new}} = \text{clamp}(\text{round}(8 - 2.1), 2, 10) = 6$ cores (throttle 8 $\rightarrow$ 6).

$K_p=0.02$ $K_i=0.0002$ $L_{\text{target}}=150$ ms $T=5$ s $\text{deadband}=10$ ms $\text{cooldown}=10$ s $C \in [2, 10]$

Report §3.5.2 (PI law).

Actuation: in-place pod resize

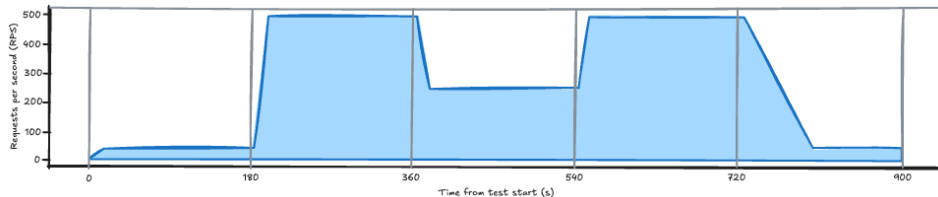
Change the Ray worker CPU limit **without restarting the container**, via the Kubernetes pods/resize subresource (Beta in k8s 1.34, GA in 1.35):

- No restart → preserves the live DDP session and pip cache.
- Python client lacks the subresource → controller sends a **raw HTTPS PATCH**.
- limits & requests set together → pod stays **Guaranteed** QoS.
- RBAC: pods/resize is a **separate resource** → 403 if omitted.

```
{"spec": {"containers": [{"name": "ray-worker",  
  "resources": {"limits": {"cpu": "N"}, "requests": {"cpu": "N"}}}]}}
```

In-place resize: Kubernetes docs (2025); cgroup v2 / CFS: Linux kernel docs. Report §3.5.3.

Experimental design



5 conditions (web-alone, AI-alone, no-limits, static 6-core, qos-controller) $\times$ **3 runs**, 15 min each. WaveShape:
30 $\rightarrow$ 400 $\rightarrow$ 200 $\rightarrow$ 400 $\rightarrow$ 30 users.

Hypothesis	Statement
H1a	no-limits raises P99 vs isolated web (noisy neighbor is real)
H1b	static limits reduce P99 vs no-limits
H2	controller SLO compliance $\geq$ static limits
H3	controller makespan $<$ static limits (reclaims idle CPU)

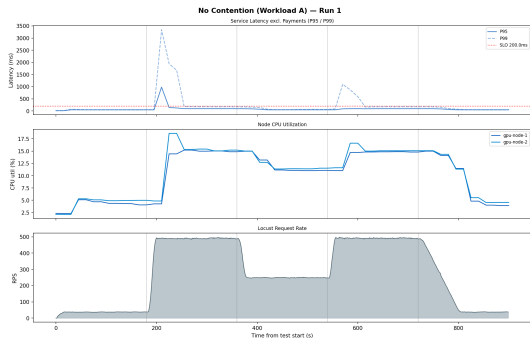
Motivation & Problem

Approach

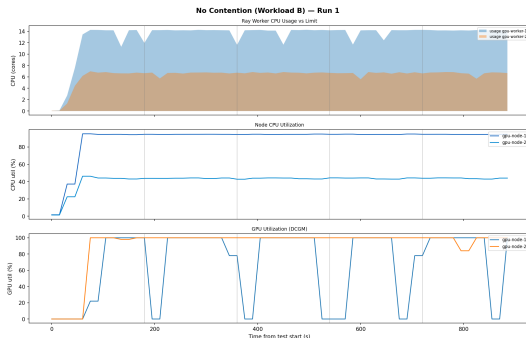
Results

Discussion & Conclusion

Baselines: the floor and the ceiling



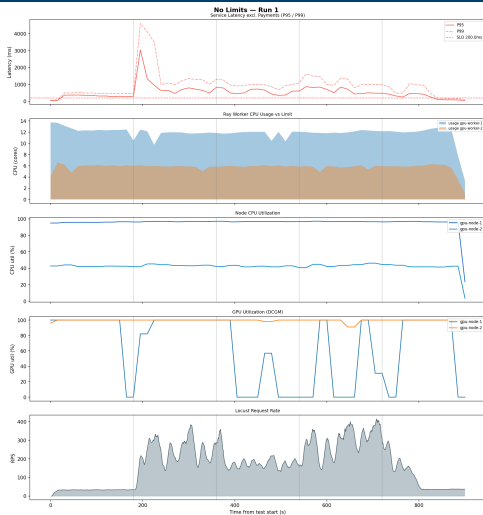
Web alone, the latency floor. P99 **273 ms**, **89.9 %** compliance. The two P99 spikes are bcrypt signup bursts → an **application-level ceiling** no CPU policy can beat.



AI alone, the throughput ceiling. Makespan **888 s**, the best-case (fastest) training: full 16 cores, no co-tenant to contend with.

Isolation baselines bracket all co-located results. Report \$4.1.

No limits = worst case

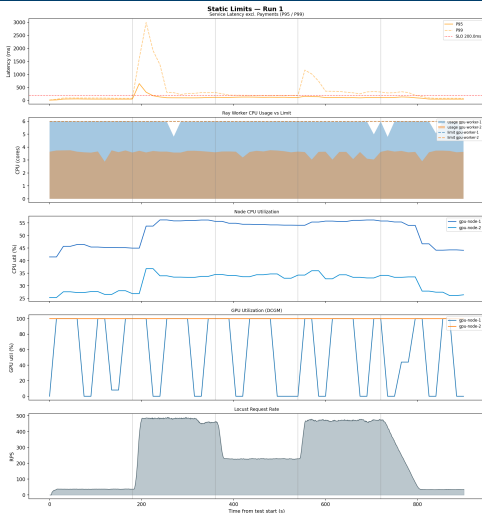


Ray unrestricted (16 cores):

- P99 **906 ms** ($3.3\times$ baseline); compliance **12 %**; violations $\approx 89\%$ of the run.
- Fails from just **30 users** (Phase 1), the only condition failing below peak; even Phase 3 never recovers.
- Client harmed: RPS **-38%**; failures **40.5 %** vs 5.1 %.
- **Asymmetry of harm**: AI fine, web wrecked.
- Makespan $1040\text{ s}^\dagger$ **inflated** (job ended before Locust).

Report §4.2.

Static limits: protection, but waste

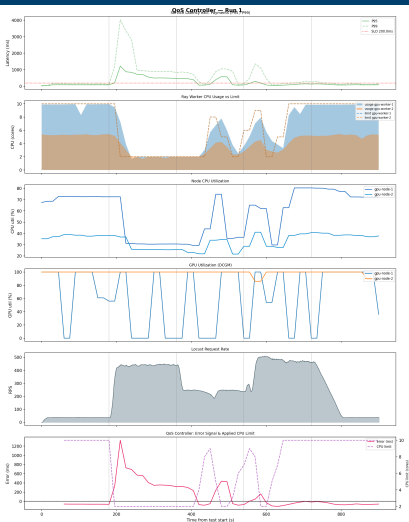


Fixed **6-core** cap:

- Compliance **49.1%**; makespan **1890 s** (+113% over the 888 s ceiling).
- Remaining spikes are bcrypt, **not Ray**.
- Per-worker asymmetry: W1 (3090) saturates 6 cores; W2 (3080) only ≈ 3.6 ; the faster GPU drives more preprocessing CPU.
- Node CPU only **41.6 %** → **stranded CPU** in low-load ($\approx 60\%$ of the test).
- Web throughput protected ($\approx$ baseline).

Report \$4.3; stranded CPU → \$5.4.

Dynamic QoS controller: the limit follows the load



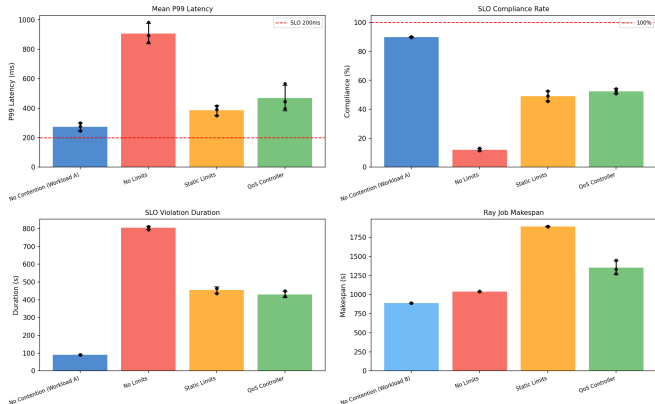
Limit tracks load **10**→**2**→**10** cores:

- Phase 1: held at **10**. Phase 2: bcrypt drives P95 >900 ms → **clamp 10**→**2** in **one step**.
- Phase 3: release 2 → 4 → 8 → 9; Phase 4: clamp, then **faster** recovery (integral wound down) → 10 by Phase 5.
- ≈16 adjustments/worker; error vs applied limit move in **opposition** (bottom panel).
- Node CPU **50.6%**; makespan **1351s**.
- Compliance **52.5%**, **lower variance**; residual violations = bcrypt.

Report §4.4.

Cross-condition comparison

Cross-Condition Comparison



Report §4.5.

vs **static limits**, both objectives at once:

- **+3.4 pp** compliance (52.5 vs 49.1 %), **lower variance** (± 1.7 vs ± 3.5).
- **-29%** makespan (1351 vs 1890 s); node CPU 41.6 $\rightarrow$ **50.6 %**.
- GPU steady **$\sim 80\%$** across all conditions regardless of cap (prefetch queue keeps it fed).
- Floor is the ceiling: web-alone **89.9 %** caps everyone (bcrypt); the rest is shared-hardware cost.

Both objectives, one feedback loop

+3.4 pp

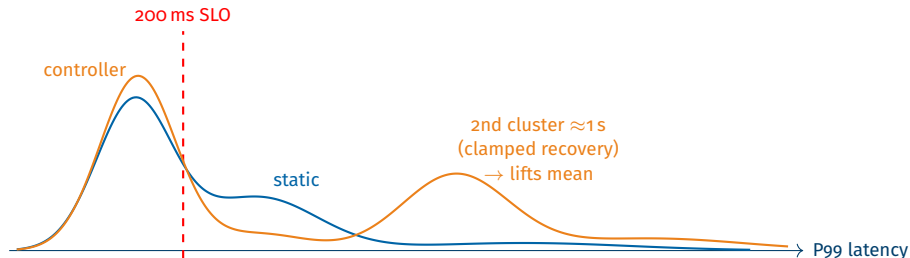
SLO compliance vs static limits
(52.5 % vs 49.1 %, lower variance)

-29%

Ray training makespan
(1351 s vs 1890 s)

The nuance to pre-empt

Controller mean P99 (**468 ms**) is *higher* than static (**385 ms**), yet its compliance is **better** (52.5 vs 49.1 %). Two statistics of the same run:



Both keep about the same mass *left of the line* → compliance is comparable. The difference is on the *right*: static's violations sit lower (200–500 ms); the controller's form a **second cluster of ≈ 1 s windows** (clamped recovery) that lifts its **mean** P99 while aggregate violation time still *falls* (430 vs 455 s).

P99 distributions, static vs dynamic (run-1 p99_latency_ms.csv).

Motivation & Problem

Approach

Results

Discussion & Conclusion

Hypotheses → research question answered

- ✓ **H1a** (*the noisy neighbor is real*) → **confirmed**: no-limits compliance **12 %**, fails even at **30 users**.
- ✓ **H1b** (*static limits reduce P99*) → **confirmed**: restores compliance below the **89.9 %** bcrypt ceiling.
- ✓ **H2** (*controller SLO $\geq$ static*) → **met**: controller $\geq$ static, with **lower variance**.
- ✓ **H3** (*controller faster than static*) → **met**: makespan **-29%**, reclaims stranded cores.

RQ answered: yes, one PI loop reclaims idle CPU *and* protects the SLO, fully in-cluster.

Caveat: +3.4 pp margin bounded above by the 89.9 % application-level ceiling (bcrypt spikes).

Three structural limits of a single-signal reactive loop:

- **Signal conflation:** one P95 signal can't separate Ray contention from app-internal bcrypt; at phase transitions it throttles Ray to $C_{\min} = 2$ for **no SLO benefit**, needlessly starving it.
- **Measurement-window dilution** (**~15–30 s** reaction): the 60 s rate() window dilutes a fresh spike with pre-spike history, so the error looks smaller than it is during an onset, while P99 can already breach 200 ms. **Observation**, not actuation, dominates (cgroup write is ms).
- **Recovery lag:** after a clamp to $C_{\min}=2$, the limit doesn't jump back; it ramps up over ~60–90 s (a few 5 s steps, gated by the 10 s cooldown), so Ray is briefly under-served at the start of each low-load phase. Bounded, not multi-minute: anti-windup freezes the integral while saturated.

Report §5.3 (limitations), §5.5 (reaction time).

Future work

- **Predictive control:** short-horizon RPS forecast → **pre-throttle** before the spike, closing the 15–30 s gap.
- **Latency signal decomposition:** per-service PromQL to suppress the Ray throttle when the rise is bcrypt-only → fixes the signal-conflation limit.
- **eBPF measurement + actuation:** kernel-space P95 + direct `cpu.max` write, bypassing the API path → **sub-second** signal-to-enforcement.
- **Multi-resource actuation:** also shape memory (`memory.max`) and network (AllReduce traffic); one PI accumulator per dimension.

Report §5.7 (future work).

A single, simple, in-cluster PI feedback loop protects a web app's SLO while cutting AI training time 29 % vs a static cap.

Needs only a latency signal and a CPU throttle · runs entirely in-cluster, so data never leaves.

Thank you! Questions?